Custom Theremin Analog Synth + Frequency Response

Arnav Sharma

Origins of the Project

A theremin is a musical instrument that can be played without actually touching it. It consists of two antennas, one of which controls the pitch based on how close you are to it and the other which controls the volume, also based on distance. I had two electronic components that gave out differing electronic signals based on how close you are: a photoresistor, which increases resistance as your hand covers it due to less light, and an ultrasonic sensor, whose ECHO pin goes HIGH for a lower period of time as your hand covers it due to the decreasing distance. I also had a passive piezo buzzer which could emit different frequencies. With this, I could create a theremin circuit of my own.

With the use of an ESP32, doing this would certainly be easier. I could use the photoresistor as a part of a voltage divider, connected to an ADC pin, to measure how much it's covered and directly update the piezo buzzer frequency. I could also directly trigger the ultrasonic sensor and measure the exact distance. However, I wanted to make this project a bit more challenging, so my goal became to build the theremin as an analog synth, without the use of any code. Any ESP32 in this project was either used for the second part of this project, and/or for just powering the circuit.

I also want the user to know what frequency they are generating. There is an ML model called CREPE that can input audio waveforms and estimate the frequency. So, the second part of this project became recording the theremin output, running the model on the ESP32, and reporting the frequency to the user. In my last project, the device detection system, the ESP32 sent data to the Flask server to run the model. I wanted to do the alternative for this project, which is deploying and running the model directly on the ESP32, no WiFi needed.

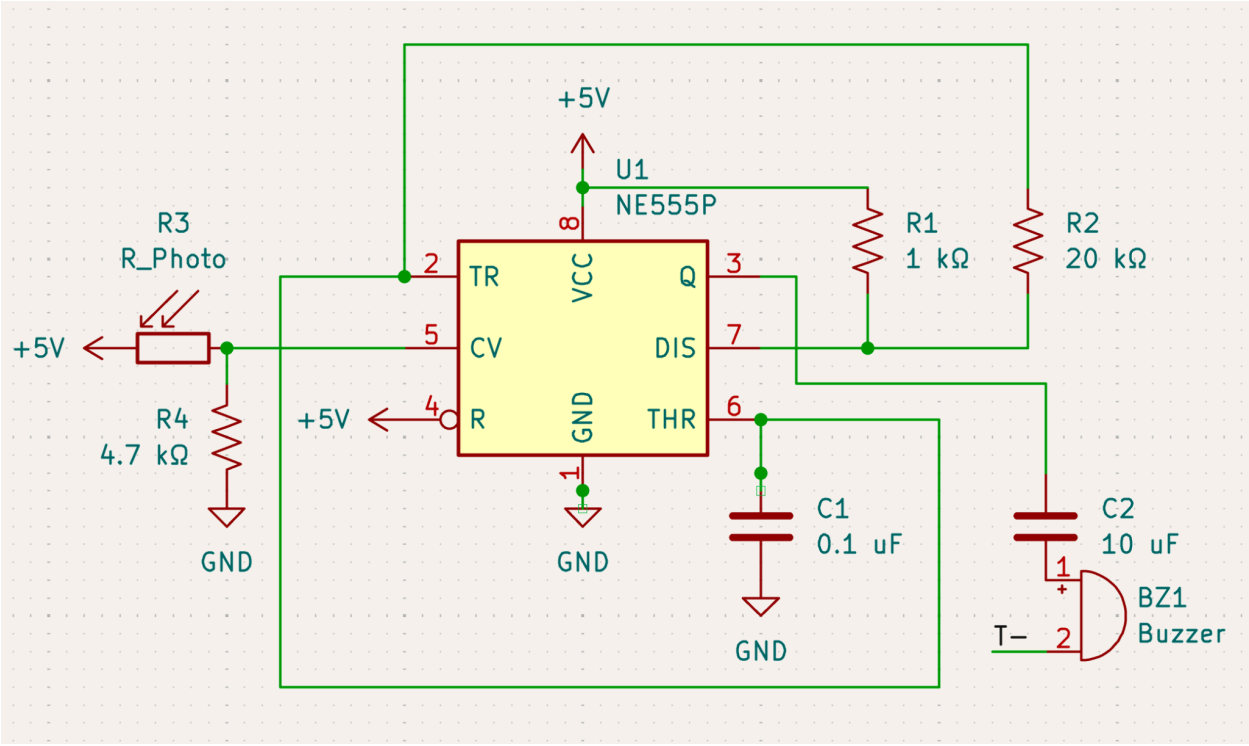
Part 1: Theremin Analog Synth

Photoresistor-controlled Pitch

To start off, I needed to place this photoresistor as a part of a voltage divider such that the output voltage would vary based on the distance. The question is, how would this output voltage lead to a change in frequency for the buzzer?

To do this, we will rely on a 555 timer, namely the NE555P. This timer can output an oscillating digital waveform in astable mode, and by feeding this waveform into the buzzer, we can vary the buzzer frequency by varying the waveform frequency. The NE555P has a "Control Voltage" pin, which can vary the upper voltage threshold of the timer. So, when decreasing the control voltage through the photoresistor's voltage divider, the output frequency increases, and vice-versa. To calculate the frequency outputted by a 555 timer, the formula as per the NE555P datasheet is $F = 1.44 \div ((R1 + 2R2)C1)$. With R1 = 1k Ω, R2 = 20 kΩ, and C1 = 0.1 uF, F = 351 Hz, it'll vary frequency around where the typical piano varies. I'll also connect the output pin to a 10 uF capacitor before connecting it to the buzzer, such that current flows only when the signal is actively oscillating.

The full schematic for this part looks like this(done on KiCAD):



(Note: The T- label for the negative terminal of the buzzer will be explained later)

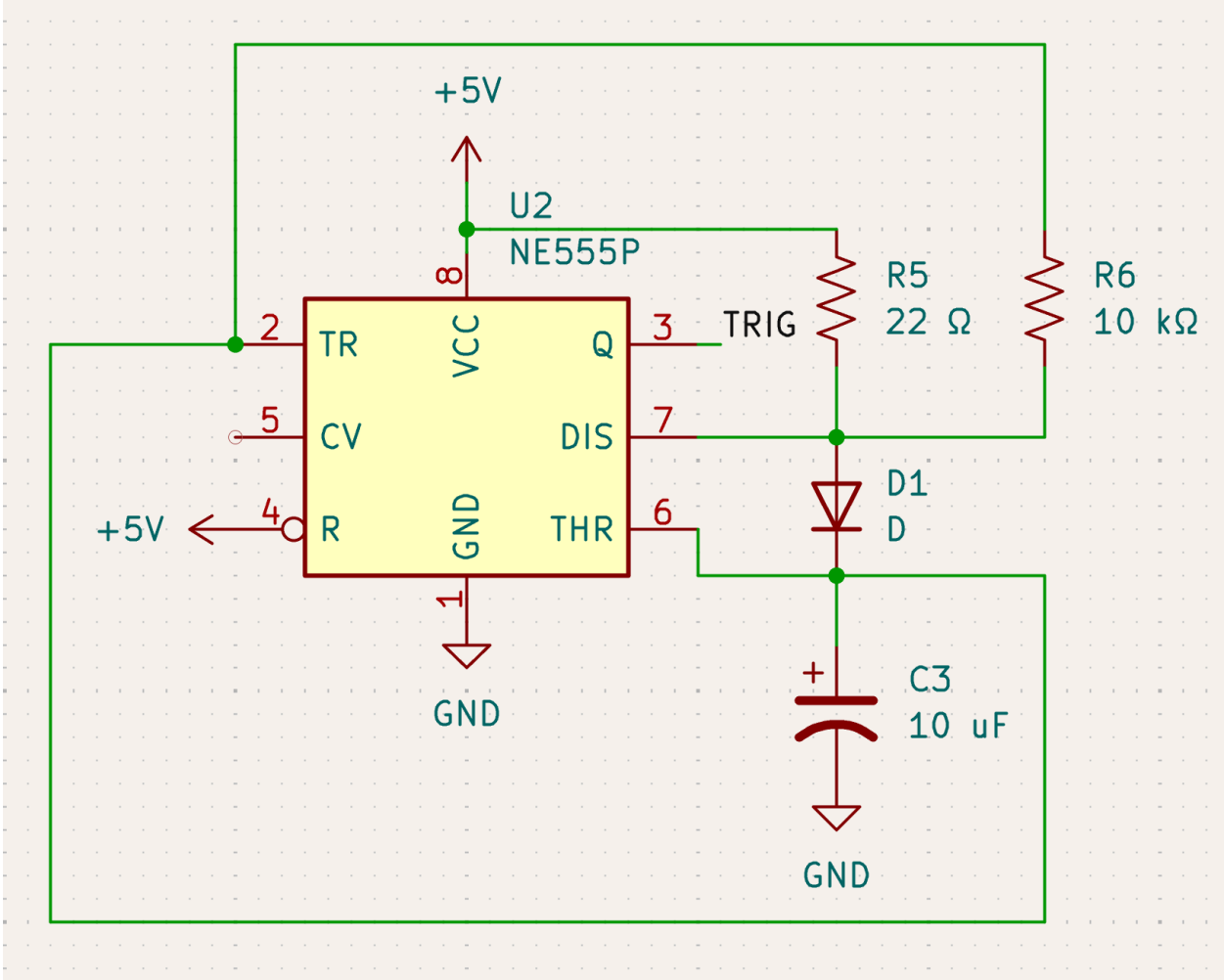
Ultrasonic sensor

The first step of completing the circuit for the ultrasonic sensor is triggering the TRIG pin, which is required for the sensor to transmit waves. Triggering the TRIG pin means setting it HIGH for a short period of time, then LOW for a much longer period of time, and continuously repeating this. For this project, I aimed for the TRIG pin to be set at HIGH for 0.1 ms, and LOW for 60 ms.

We can use another NE555P for this. By configuring the resistor and capacitor values, the time that the output wave is at HIGH and LOW can be changed. The exact formulas on the NE555P datasheet are T(high) = 0.693*(R1+R2)*C, and T(low) = 0.693*R2*C. There is one issue: T(high) is always >= T(low), because T(high) depends on R1 and R2, but this is the opposite of what we want.

In order to eliminate R2 from the equation, we need to make sure that the NE555P's charging path goes through R1 only, and the discharging path goes through R2 only. To do this, we can use a diode, with the cathode connected to the timing capacitor, and the anode connected to pin 7. This way, for the charging path, the diode offers a 0 resistance path such that current skips R2, while for the discharging path, the diode path is infinite resistance, so the current has to go through R2. From here, R1 = 22 Ω, R2 = 10 kΩ, and C = 10 uF leads to T(high) = 0.693*R1*C = 0.15 ms, and T(low) = 0.693*R2*C = 69.3 ms.

The full schematic is below:



Unlike pitch, controlling the volume of the buzzer is less direct. One way to control it is through current. If you limit the current coming to the buzzer, the volume will decrease, with less current meaning less volume. So the question becomes how to decrease the current coming to the buzzer with the ECHO pin output of the ultrasonic sensor.

One way to do this is with a transistor. Specifically, the transistor I chose is the 2N5088 BJT transistor. This transistor has a collector, emitter, and a base. For an NPN, which is what the 2N5088 is, current flows from the collector to the emitter, and the base acts like a gate. As you increase the voltage applied to the base, the base lets more and more current through to the emitter until it is fully open. Based on the 2N5088 datasheet, the base-emitter voltage is 0.8 V, meaning applying 0.8 V to the base will lead to it fully opening up. So, a little lower than this, around 0.6 V - 0.8 V, is when the base is most sensitive to changes in voltage.

I connected the transistor to the circuit with the buzzer by simply connecting the emitter to GND and the collector to the - terminal of the buzzer, removing the old GND connection. This way, current goes through the + buzzer terminal, to the - buzzer terminal, through the collector, to the base, to the emitter, and finally to GND.

The ECHO pin of the ultrasonic sensor can be used as a general gauge of distance, but its output is quite complex. The pin stays at LOW until it gets triggered by the TRIG pin and sends the wave, in which it will stay HIGH until the sensor detects the wave bouncing back. The longer the ECHO pin stays at HIGH, the more distance there is to the nearest object. This creates a dilemma I initially faced: If you connect this pin to a resistor and then to the transistor's base, the base will open up when ECHO goes HIGH, but close when ECHO goes LOW, repeating over and over again. So, while the buzzer did generally go quieter as the ultrasonic sensor was covered, the buzzer output was not clean, and instead was constantly alternating between LOW and HIGH in varying frequencies.

I had to transform the ECHO output from an alternating voltage to a more steady voltage that still changes depending on the distance detected. To use this, I made an operational amplifier integrator with an MCP6002 op-amp. The output voltage of this integrator is proportional to the integral of the input voltage over time. Essentially, it transforms a square wave into a triangular wave, which is much more gradual/steady. If you apply a resistor in parallel to the capacitor of the integrator, it will prioritize later voltages, so it won't just keep increasing as time goes on.

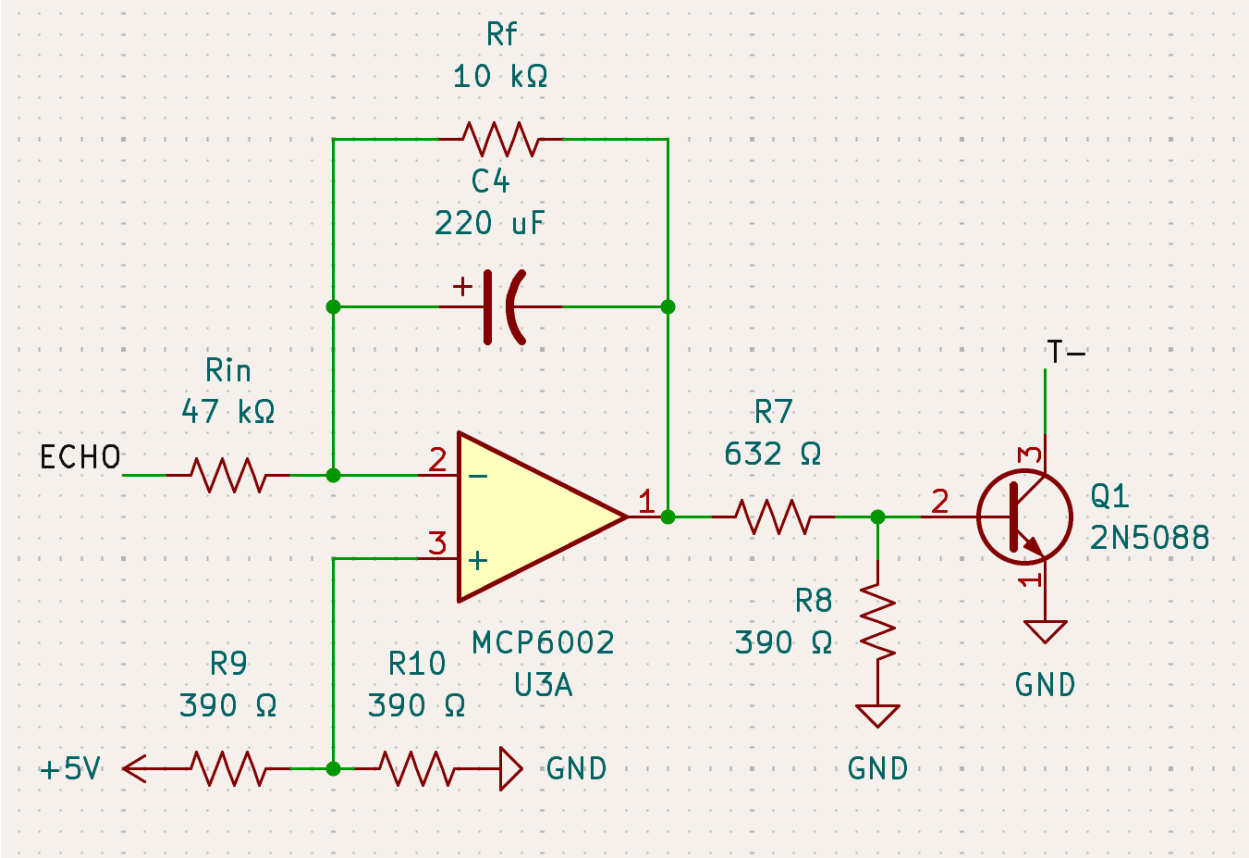
Deciding on the values of the two resistors and capacitor was not an easy task. There are two main important formulas. The cutoff frequency, which is the frequency at which the op-amp starts becoming an integrator, is f = 1/(2*pi*Rf*C). The gain formula changes depending on whether the integrator is integrating or not, but is always proportional to 1/Rin. This means that as you increase Rin, the output voltage changes less as the input voltage changes, and vice-versa. After a lot of trial and error, I ended up with Rin = 47 kΩ, Rf = 10k Ω, and C = 220 uF.

The op-amp integrator requires a negative power source as well as a positive voltage source, as the input voltage goes to the inverting input(-) of the op-amp. However, you can technically evade this by using a voltage divider to connect 2.5 V to the non-inverting input(+), so the output can now swing in both ways, either above or below the virtual ground of 2.5 V.

Because the output voltage initially was significantly higher than the 0.6 - 0.8 V range that is needed for the base, I connected Pin 1 of the MCP6002, which is the output pin to a

voltage divider to get an output within that range. The voltage divider consists of the first resistors making up 632 Ω and the resistor to GND of 390 Ω.

The full schematic is below:



(Note that T- refers to the - terminal of the piezo buzzer)

Given how complicated this all is compared to the photoresistor, you might be wondering why I didn't just get another photoresistor and use that instead of an ultrasonic sensor for controlling pitch. But where would be the fun in that? :)

Part 2: Pitch-Detection Model

Deploying the Model

For this part of the project, I had to find an AI model that could detect pitch from audio, and then deploy it directly on my ESP32. The ESP32-C3 has 4 MB of flash and 400 KB of SRAM, so I had to make sure the model fits these constraints. I ended up going with the CREPE model, an AI pitch-detection model that has various sizes(tiny, small, medium, etc..).

The small model has 1.6 million parameters and takes up 6.2 MB of space, which is above the amount of flash on the ESP32. Using Tensorflow's TFLiteConverter, I int8 quantized this model, reducing the space to a 1.7 MB .tflite file.

When quantizing a model, there are important details to note. For one, you have to get the data on the scales for the input and the output, which are pretty much the quantization parameters in which you convert whatever you're inputting into the model from a floating point datatype to an int8, and the way you convert the int8 output back into a floating point datatype. The formulas are value / INPUT_SCALE + INPUT_ZERO_POINT for the input, and (value - OUTPUT_ZERO_POINT) * OUTPUT_SCALE for the output. Also, in order to actually upload the model to the ESP32 through Arduino IDE, the .tflite file is not enough. Instead, you first need to convert the .tflite file to a C header file. You can use the command line to do this through the command "xxd -i model.tflite > model.h" on Linux or MacOS. This header file holds a character array representation of the model.

Loading the model header file and running the model is a multi-step process. You need to pass your model into a MicroInterpreter object, which actually runs it. You also need to decide on the maximum amount of SRAM that this object can use to run the model, and pass in a dedicated SRAM block. There is not a precise way to calculate this value, so after some trial and error I ended up allocating 150 KB of SRAM.

From this MicroInterpreter, you can get pointers to the input and output tensor. This is how you actually run the model. The CREPE model takes 1024 inputs of audio and has 360 outputs, each representing a probability of a specific pitch. For each input, you must quantize it into int8 before placing it into the input tensor. You then need to invoke the MicroInterpreter, which runs the model. Then, for each output that you get from the output tensor you have to dequantize it to get the actual probability.

Recording Audio

I then needed a microphone to actually record audio to be inputted into the model. At first, I tried a microphone module with an elecret and op-amp, but I found that the output changes were minimal when the buzzer changed pitch. So, I ended up going with an INMP441 microphone. This microphone sends data to the ESP32 via a communication protocol known as I2S, which is meant specifically to send audio data from one integrated circuit to another.

Connecting the microphone to the ESP32 is relatively simple. The VDD of the mic connects to the 3.3 V of the ESP32, because as per the INMP441 datasheet, the maximum supply voltage is 3.63 V. They share a common GND connection, and L/R can also be set to the

GND, which means the INMP441's data goes through the left channel. The other pins can be connected to any GPIO pin on the ESP32. According to the datasheet, the mic has an internal ADC converter, so connecting these pins to ADC pins is unnecessary. I also added a 0.1 uF decoupling capacitor(filters out high frequencies) between VDD and GND, as this was recommended by the datasheet.

In the Arduino IDE, I needed to configure the I2S. Since the CREPE model is trained on audio data at a 16 kHz sampling rate, I made the sampling rate as such. I also decided to record 16 bits per sample, meaning 8 bits were dropped from the standard 24 bits per sample that the INMP441 records(as per the datasheet), in order to save on SRAM. It was also here that I needed to instruct the code to check audio data from the left channel due to the L/R to GND wiring. After setting up the I2S, simply running the i2s_read command writes the data to an array consisting of 1024 16 bit integers. I could then feed this array into the model to get and print out the predicted frequency.

Using FreeRTOS

The most basic way to cycle through audio recording and model inference is to just do them one after the other. However, one major issue with this is that while the model is running, the microphone is not recording any samples. So, I used FreeRTOS in order to ensure as much audio was captured as possible.

For one, I used a queue with a length of 4 as the means of storing audio data. I created two tasks: One for the mic, which continuously called on i2s and then sent the audio data to the queue, and one for the model, which continuously pulled data from the queue and ran the model on it. I made the model task a higher priority than the mic task, because the model took a lot more time to run, and making the mic task higher resulted in the model rarely outputting. Now, while the model is running the inference, the queue can accumulate data up until a length of 4 rather than not collecting anything.